

INTERVIEW PREPARATION: SENTIMENT ANALYSIS ON IMDB USING WORD2VEC

Project: Sentiment Analysis on IMDB Dataset using Word2Vec and Deep Learning
Author: Shubham Soni (CS22B1053), IIIT Raichur
Semester: 6th Sem Mini Project | Supervisor: Dr. Natesha B V

PART 1: PROJECT EXPLANATION SCRIPT (2 Minutes)

"My 6th semester mini project is Sentiment Analysis on the IMDB Movie Reviews dataset using Word2Vec embeddings and Deep Learning.

The goal is binary classification - given a movie review, predict whether it's positive or negative. I used the standard IMDB dataset of 50,000 reviews, split 80-20 for training and testing.

The pipeline has three stages. First, TEXT PREPROCESSING - I clean the reviews by removing HTML tags, fixing contractions, removing stopwords, and applying lemmatization to reduce words to their base form.

Second, WORD EMBEDDINGS - I use Google's pre-trained Word2Vec model which represents each word as a 300-dimensional vector. This is a form of transfer learning - the model was trained on 100 billion words from Google News, so it has rich semantic understanding that benefits our smaller task.

Third, DEEP LEARNING MODELS - I implemented and compared 5 architectures: CNN, LSTM, Bi-LSTM, 2-Layer Bi-LSTM, and a CNN-LSTM Hybrid.

The key finding is that CNN performed BEST at 89.02% accuracy. This makes sense because sentiment in movie reviews is primarily determined by local phrases like 'waste of time' or 'absolutely brilliant' - and CNN excels at detecting these local patterns through its convolutional filters. LSTM-based models performed slightly lower (87-88%) because long-range sequential dependencies matter less for this binary sentiment task.

All models used the same Word2Vec embeddings and preprocessing, so the comparison is fair and shows that architecture choice impacts performance by about 1-2%."

PART 2: SHORT VERSION (30 Seconds)

"I built a sentiment analysis system for IMDB movie reviews using Word2Vec embeddings and deep learning. I compared 5 architectures - CNN, LSTM, Bi-LSTM, 2-layer Bi-LSTM, and CNN-LSTM hybrid. CNN achieved the best accuracy of 89.02% because sentiment is mostly determined by local phrase patterns, which is exactly what convolutional filters capture. The preprocessing pipeline includes HTML removal, stopword elimination, and lemmatization, and I used Google's pre-trained Word2Vec for 300-dimensional word representations."

PART 3: INTERVIEW QUESTIONS & ANSWERS

SECTION A: TEXT PREPROCESSING QUESTIONS

Q1: Why do you need text preprocessing? Can't you feed raw text to the model?

A: Neural networks need NUMERICAL input - they can't process raw strings. But beyond that, preprocessing removes NOISE. Raw IMDB reviews have HTML tags from web scraping, inconsistent capitalization, stopwords that appear in every review regardless of sentiment, and different word forms (liked/liking/likes). All of these add noise without adding signal. Preprocessing gives cleaner, standardized input that helps the model focus on what actually indicates sentiment.

Q2: Why remove stopwords? Don't they carry some meaning?

A: Stopwords like "the", "is", "a", "an" appear equally in positive AND negative reviews - they don't discriminate between classes. Removing them reduces sequence length (less padding needed), reduces noise, and lets the model focus on content words that carry sentiment ("terrible", "amazing", "boring"). However, there's a trade-off: "not" is often in stopword lists but is crucial for sentiment ("not good" vs "good"). In our preprocessing, we're careful about which stopwords to remove.

Q3: Why lemmatization over stemming?

A: Stemming just chops endings: "studies" -> "studi" (not a real word). Lemmatization uses dictionary lookup: "studies" -> "study" (valid word). This matters because Word2Vec has vectors for real words like "study" but not for stems like "studi". Lemmatization ensures maximum coverage in the pre-trained Word2Vec vocabulary, meaning fewer OOV (out-of-vocabulary) words that get zero vectors.

Q4: Why pad/truncate to exactly 300 tokens?

A: Neural networks need fixed-length input because the weight matrices have fixed dimensions. 300 was chosen because: (1) it covers the majority of reviews without excessive truncation - most reviews are under 300 words after preprocessing; (2) it's not too long - longer sequences slow training and risk the LSTM forgetting early information; (3) it's a common standard in NLP literature for this dataset. Short reviews are zero-padded, long ones are truncated to the first 300 tokens.

Q5: What happens to the meaning when you remove punctuation and uppercase?

A: We lose some information. "TERRIBLE!!!" carries more emphasis than "terrible" - the caps and exclamation marks signal strong negative sentiment. However, Word2Vec doesn't have separate embeddings for "TERRIBLE" vs "terrible" or "terrible!!!", so keeping them would just create OOV words. The trade-off is worth it: cleaner input with better embedding coverage outweighs the lost emphasis information for most cases.

SECTION B: WORD2VEC QUESTIONS
#####

Q6: What is Word2Vec? How does it work?

A: Word2Vec is a shallow neural network that learns word vector representations by predicting words from their context. It has two variants: Skip-gram (predicts surrounding words given the center word) and CBOW (predicts center word given surrounding words). The key insight is that words appearing in similar contexts have similar meanings, so they get similar vectors. "good" and "great" appear

in similar contexts ("the movie was good/great") so they end up with similar 300D vectors.

Q7: Why use PRE-TRAINED Word2Vec instead of training your own?

A: Google's pre-trained model was trained on 100 BILLION words from news articles. Our IMDB dataset has only 50,000 reviews (~10 million words). Training Word2Vec from scratch on 10M words would give poor embeddings - rare words that appear only 2-3 times would have unreliable vectors. The pre-trained model has seen millions of contexts for each word, giving much richer representations. This is transfer learning - leveraging knowledge from a massive unlabeled corpus.

Q8: What are out-of-vocabulary (OOV) words? How do you handle them?

A: OOV words are words in our reviews that don't exist in the Google News Word2Vec vocabulary. Examples: movie-specific slang ("MCU", "plot hole"), misspellings ("awsome"), very informal language ("lolll"). We map these to ZERO vectors - effectively making the model ignore them. This is a limitation. Alternative: FastText handles OOV better by using subword information (breaking "plot hole" into "plot" + "hole"), but we chose Word2Vec as the standard approach.

Q9: Why 300 dimensions? Why not 50 or 1000?

A: 300 is the standard for Google's pre-trained Word2Vec model - it's what's available. But the reasoning behind 300: fewer dimensions (50) can't capture the full semantic richness of language - too much information compressed. More dimensions (1000) would be sparser, harder to train downstream models, and provide diminishing returns. Research has shown 200-300 dimensions is the sweet spot for most NLP tasks - enough capacity to represent nuanced meaning without overfitting.

Q10: What's the difference between Word2Vec and one-hot encoding?

A: One-hot: each word is a sparse binary vector of vocabulary size (e.g., 50,000 dimensions, with only one "1"). Problems: huge dimensionality, no semantic similarity (all words equidistant), no relationship captured.

Word2Vec: each word is a dense 300D vector. Benefits: low dimensionality, semantically similar words have similar vectors ("good" close to "great"), captures relationships (king-man+woman=queen). Word2Vec makes downstream models much more effective because the input already carries meaning.

Q11: You said the embedding layer is "frozen". What does that mean?

A: Frozen means the Word2Vec weights are NOT UPDATED during training. The embedding layer is loaded with pre-trained vectors and set to trainable=False. The model learns to USE these embeddings but doesn't modify them.

WHY freeze: (1) Pre-trained vectors are already high-quality from 100B words;
(2) Updating them on only 40K reviews might damage the general knowledge;
(3) Reduces trainable parameters, preventing overfitting.

Alternative: Fine-tuning (trainable=True) can sometimes improve results if the domain is very different from the pre-training corpus, but for English movie reviews + Google News embeddings, freezing works well.

Q12: What is the difference between Word2Vec and BERT embeddings?

A: Word2Vec gives STATIC embeddings - the word "bank" always has the same vector regardless of whether it means "river bank" or "financial bank". BERT gives CONTEXTUAL embeddings - "bank" gets different vectors based on surrounding words.

BERT is more powerful (93-95% on IMDB vs our 89%) but requires 110M parameters, GPU, and much more computation. For a 6th-sem project demonstrating NLP concepts, Word2Vec + deep learning is the appropriate level of complexity while still achieving competitive results.

SECTION C: MODEL ARCHITECTURE QUESTIONS
#####

Q13: Why did CNN perform best? Isn't LSTM supposed to be better for text?

A: Common misconception! LSTM is better when LONG-RANGE DEPENDENCIES matter - like machine translation where a word at position 50 depends on position 5. For binary sentiment analysis, the decision is mostly based on LOCAL PHRASES: "terrible movie", "waste of time", "absolutely loved it". CNN's convolutional filters are DESIGNED to detect exactly these local n-gram patterns. LSTM processes sequentially and may "forget" early patterns by the time it reaches the end of a 300-word review. CNN captures all patterns in parallel and takes the strongest signals via max pooling.

Q14: What is a Conv1D layer doing on text?

A: Conv1D applies sliding filters over the word sequence. If we have a filter of size 3, it looks at 3 consecutive word vectors at a time (300D each), producing one scalar output. This is like a trainable trigram detector. With 128 filters of size 3, we get 128 different pattern detectors, each learning to recognize a different meaningful 3-word pattern. After convolution, max pooling picks the STRONGEST activation across the entire review for each filter - meaning "if this pattern exists ANYWHERE in the review, capture it."

Q15: How does LSTM handle the vanishing gradient problem?

A: Regular RNNs multiply gradients through many time steps during backpropagation - if multiplications are <1, gradients vanish; if >1, they explode. LSTM solves this with a CELL STATE that flows through time with only additive interactions (no multiplication). The three gates (forget, input, output) control what gets added or removed from the cell state. This additive flow preserves gradients over long sequences, allowing LSTM to remember information from 100+ steps ago.

Q16: Why did Bi-LSTM perform WORSE than regular LSTM (87.63% vs 88.05%)?

A: Counterintuitive but explainable. Bi-LSTM has DOUBLE the parameters of LSTM (forward + backward). With our fixed training data of 40K reviews, the extra parameters lead to slight overfitting. Also, for sentiment analysis, forward-only reading is often sufficient - you typically build up opinion as you read left to right. The backward pass adds parameters and computation without proportional benefit for this specific task. With more data or stronger regularization, Bi-LSTM could potentially outperform.

Q17: What's the advantage of 2-Layer Bi-LSTM over single Bi-LSTM?

A: Stacking creates a HIERARCHY. The first layer captures low-level patterns

(word-level and phrase-level features). Its output is a new sequence of learned representations. The second layer captures higher-level patterns (sentence-level relationships) from these learned representations. It's similar to how stacking CNN layers captures increasingly abstract features. The 2-layer model (88.52%) indeed outperforms single Bi-LSTM (87.63%) by ~1%.

Q18: In CNN-LSTM hybrid, what's the role of each component?

A: CNN acts as a FEATURE EXTRACTOR - its filters detect local patterns (phrases). Max pooling produces a downsampled sequence of detected features. LSTM then processes this feature sequence SEQUENTIALLY - understanding how these local patterns relate to each other across the review. Think of it as: CNN says "here are the important phrases" and LSTM says "how do these phrases flow together?" In practice, for binary sentiment, the sequential flow between phrases matters less than just HAVING them, which is why pure CNN still wins.

Q19: What is dropout and where did you use it?

A: Dropout randomly sets a fraction (e.g., 20-50%) of neuron outputs to zero during TRAINING. This prevents neurons from co-adapting (relying on specific other neurons) and forces the network to be robust - like training an ensemble of subnetworks. During INFERENCE, dropout is turned off and all neurons are used. I applied dropout between the model output and the final Dense layer to prevent the classifier from overfitting to specific feature combinations.

Q20: Why sigmoid activation at the output? Why not softmax?

A: Sigmoid maps output to [0, 1] representing P(positive). If output > 0.5, classify as positive. Softmax is used for MULTI-CLASS problems (outputs sum to 1 across N classes). Since we have binary classification (2 classes), a single sigmoid neuron is sufficient and equivalent to a 2-class softmax. Using sigmoid with binary crossentropy loss is the standard approach for binary classification.

Q21: What is binary crossentropy loss?

A: The loss function for binary classification. For a single prediction:
Loss = $-[y \cdot \log(p) + (1-y) \cdot \log(1-p)]$

If true label $y=1$ (positive) and model predicts $p=0.9$: Loss = $-\log(0.9) = 0.1$ (low, good)
If true label $y=1$ (positive) and model predicts $p=0.1$: Loss = $-\log(0.1) = 2.3$ (high, bad)

It penalizes confident wrong predictions severely. The model minimizes the average loss across all training samples by adjusting its weights.

SECTION D: RESULTS AND EVALUATION QUESTIONS
#####

Q22: 89% accuracy - is that good? What's the state-of-the-art?

A: For the context of this project, yes. State-of-the-art on IMDB: BERT/RoBERTa achieve 93-95%. Simple baselines (Naive Bayes + BoW) get ~85%. Our 89% with Word2Vec + CNN is solidly above baselines and demonstrates effective use of embeddings and deep learning without the complexity of transformers. For a 6th-sem project, achieving 89% while comparing 5 architectures shows strong understanding of the NLP pipeline.

Q23: What does AUC = 0.96 mean?

A: AUC (Area Under the ROC Curve) measures the model's ability to DISCRIMINATE between positive and negative reviews at ALL possible thresholds (not just 0.5). AUC = 1.0 means perfect separation. AUC = 0.5 means random guessing. AUC = 0.96 means: if you randomly pick one positive and one negative review, there's a 96% chance the model assigns a higher score to the positive one. It's threshold-independent, making it more robust than accuracy alone.

Q24: What does a confusion matrix show?

A: A 2x2 table for binary classification:

	Predicted Positive	Predicted Negative
Actual Positive:	TP	FN
Actual Negative:	FP	TN

TP = True Positive (correctly predicted positive)

FP = False Positive (negative review incorrectly predicted as positive)

FN = False Negative (positive review missed)

TN = True Negative (correctly predicted negative)

From this: Precision = $TP / (TP + FP)$, Recall = $TP / (TP + FN)$, Accuracy = $(TP + TN) / \text{Total}$

Q25: Why is the accuracy spread between models only 1.4% (87.63% to 89.02%)?

A: This tells us that the EMBEDDINGS and PREPROCESSING matter more than the specific architecture. All models benefit equally from the rich 300D Word2Vec representations. The ceiling is largely set by (1) the embedding quality, (2) the preprocessing quality, and (3) the inherent difficulty of the task (some reviews are genuinely ambiguous). Changing architectures gives diminishing returns once the input representation is already good.

#####

SECTION E: GENERIC / CONCEPTUAL QUESTIONS

#####

Q26: What is sentiment analysis? Give real-world applications.

A: Sentiment analysis is automatically determining whether text expresses a positive, negative, or neutral opinion. Applications: (1) E-commerce: analyzing product reviews at scale; (2) Social media monitoring: tracking brand perception on Twitter; (3) Customer support: routing complaints (negative) vs praise (positive); (4) Finance: predicting stock movements from news sentiment; (5) Politics: gauging public opinion on policies; (6) Healthcare: detecting depression from patient communications.

Q27: What is the difference between NLP and NLU?

A: NLP (Natural Language Processing) is the broad field of making computers work with human language - includes parsing, translation, generation, etc.

NLU (Natural Language Understanding) is a SUBSET focused specifically on COMPREHENSION - understanding meaning, intent, and sentiment from text.

Sentiment analysis is an NLU task because it requires understanding the MEANING behind words, not just processing them structurally.

Q28: What is transfer learning? How is it used in your project?

A: Transfer learning uses knowledge from one task to improve performance on a different (usually smaller) task. In my project: Word2Vec was trained on 100 billion words of news text (Task A: predict word from context). I transfer those learned word representations to my task (Task B: classify sentiment). The Word2Vec model has learned that "amazing" and "fantastic" are similar, that "terrible" and "horrible" are similar - this semantic knowledge helps my sentiment classifier even though it was learned from news, not movie reviews.

Q29: Can your model handle sarcasm? Why or why not?

A: Poorly. Example: "Oh great, another terrible sequel. What a masterpiece." Word2Vec gives "great" and "masterpiece" positive vectors, but the overall sentiment is negative (sarcastic). Our models detect individual word sentiments but struggle with the REVERSAL of meaning that sarcasm creates. Handling sarcasm requires understanding pragmatics and context beyond what Word2Vec + simple models can capture. Transformer models with attention are somewhat better at this, but sarcasm detection remains an open research problem.

Q30: What's the difference between your CNN for text and CNN for images?

A: Same concept, different dimensions. Image CNN: 2D filters slide over height and width of an image, detecting visual patterns (edges, shapes). Text CNN: 1D filters slide over the SEQUENCE of word vectors, detecting sequential word patterns (phrases). In images, filter size might be 3x3 pixels. In text, filter size might be 3 words (trigram). Both extract local features and use pooling for dimension reduction. The key difference: text is 1D sequential, images are 2D spatial.

Q31: What is the Adam optimizer? Why is it commonly used?

A: Adam (Adaptive Moment Estimation) combines two techniques: Momentum (uses running average of past gradients to smooth updates and escape local minima) and RMSProp (adapts learning rate per-parameter based on recent gradient magnitudes). Benefits: works well with default hyperparameters (lr=0.001), adapts to each parameter automatically, handles sparse gradients well, and converges fast. It's the de facto default optimizer for deep learning because it just works well in most situations without careful tuning.

Q32: What is overfitting? How do you detect and prevent it?

A: Overfitting = model memorizes training data but fails on new data.
DETECTION: training accuracy keeps increasing while validation accuracy plateaus or drops (growing gap between train and val performance).
PREVENTION in my project: (1) Dropout layers (randomly disable neurons); (2) Frozen embeddings (fewer trainable parameters); (3) Fixed train/test split (never test on training data); (4) Early stopping (stop training when val loss stops improving); (5) Balanced dataset (no class to overfit on).

Q33: If you had to choose only ONE model, which and why?

A: CNN. Reasons: (1) Best accuracy (89.02%); (2) Fastest training and inference (parallel computation, no sequential dependency); (3) Simplest architecture among the options (easiest to debug and deploy); (4) Most interpretable (filters correspond to n-gram patterns you can examine); (5) Proven effective for text classification across multiple benchmarks. The only scenario I'd prefer LSTM is if the task required understanding very long-range dependencies (like document summarization), which binary sentiment doesn't.

Q34: How would you deploy this model as a web application?

A: (1) Save the trained CNN model (model.save()); (2) Build a Flask/FastAPI backend that loads the model; (3) Create an endpoint that accepts review text; (4) Backend preprocesses the text (same pipeline), generates embeddings, runs inference; (5) Returns prediction (positive/negative) with confidence score; (6) Frontend (HTML/React) provides a text box for user input and displays results. For production: add input validation, rate limiting, and serve with Gunicorn + nginx.

Q35: What would you improve if you continued this project?

A: Five improvements: (1) Use BERT for 93%+ accuracy; (2) Add attention mechanism to LSTM for interpretability (see which words matter); (3) Expand to 5-class sentiment (very positive/positive/neutral/negative/very negative); (4) Add a web interface for live demo; (5) Try FastText embeddings to handle OOV words better. Also: (6) Experiment with fine-tuning the embedding layer (not freezing) to see if domain adaptation helps.

Q36: What is the difference between classification and regression?

A: Classification predicts DISCRETE labels (positive/negative, cat/dog). Regression predicts CONTINUOUS values (rating 1.0 to 10.0, price). My project is classification (binary: positive or negative). If we predicted the STAR RATING (1-10), that would be regression. The output layer reflects this: sigmoid for classification (probability of class), linear for regression (unbounded numerical output).

Q37: What is the role of the Dense layer before the output?

A: The Dense (fully connected) layer acts as the CLASSIFIER. The layers before it (CNN/LSTM) are FEATURE EXTRACTORS - they transform the 300-token input into a fixed-length feature vector. The Dense layer then learns a linear decision boundary in that feature space to separate positive from negative. Without it, the model would have no classification component. Typically: LSTM/CNN output -> Dense(128 or 64, relu) -> Dropout -> Dense(1, sigmoid).

Q38: What is the difference between training, validation, and test sets?

A: Training set (80%): model learns from this. Validation set: used during training to monitor overfitting (stop early if val loss increases). Test set (20%): used ONLY ONCE at the end for final evaluation - never seen during training. In my project, I used an 80/20 train/test split. Ideally, I'd also carve out a validation set from training (e.g., 70% train / 10% val / 20% test) for early stopping decisions.

Q39: Can this model work for languages other than English?

A: Not directly. The preprocessing (stopwords, lemmatization) is English-specific. The Word2Vec model is English. To support Hindi, for example, you'd need: (1) Hindi text preprocessing (different stopwords, stemming rules); (2) Hindi Word2Vec embeddings (or multilingual embeddings like MUSE); (3) A Hindi sentiment dataset for training. The MODEL ARCHITECTURE (CNN, LSTM) is language-agnostic - only the input processing and embeddings need to change.

Q40: What is the time and space complexity of inference for your CNN model?

A: For one review: (1) Preprocessing: $O(n)$ where n = number of words; (2) Embedding lookup: $O(300 * \text{sequence_length}) = O(300 * 300) = O(90,000)$; (3) Conv1D: $O(\text{filter_size} * \text{num_filters} * \text{sequence_length} * \text{embedding_dim})$; (4) Max pooling: $O(\text{num_filters} * \text{sequence_length})$; (5) Dense layers: $O(\text{hidden_units})$. Total: dominated by convolution, roughly $O(\text{millions of multiplications})$. In practice: <100ms per review on CPU, <10ms on GPU. Space: model is ~5-20MB depending on vocabulary size.

#####

SECTION F: TRICKY / DEEP QUESTIONS

#####

Q41: What if a review is mixed? ("The acting was great but the plot was terrible")

A: This is the challenge of binary classification. The model must decide positive or negative overall. Typically, it will classify based on which signal is STRONGER. If "terrible plot" has more weight than "great acting" in the learned features, it predicts negative. The model handles this reasonably well because CNN's max pooling captures the STRONGEST signals, and overall reviews with mixed opinions often lean one way. For truly ambiguous reviews, multi-class sentiment (adding "neutral" or "mixed") would be more appropriate.

Q42: You said CNN captures local patterns. Can it capture "not good" correctly?

A: Yes! If the filter size is 2 or more, it processes "not" and "good" together as a unit. A filter of size 2 sees the bigram "not good" as a distinct pattern different from just "good". Through training, it learns that "not good" has negative sentiment even though "good" alone is positive. This is one advantage of CNN over bag-of-words approaches which treat "not" and "good" independently. Filter size 3 can even capture "not very good" as a unit.

Q43: Why not just count positive and negative words (lexicon-based approach)?

A: Lexicon approaches (counting positive/negative words from a dictionary) are simple baselines (~75-80%) but fail on: (1) Negation: "not good" has "good" (positive word) but negative meaning; (2) Context: "unpredictable plot" is positive for a thriller but negative for a documentary; (3) Sarcasm: "what a masterpiece" might be sarcastic; (4) Degree: "good" vs "incredible" have different intensities. Deep learning learns these nuances from data rather than relying on a fixed dictionary.

Q44: How would you handle a dataset that's imbalanced (90% positive, 10% negative)?

A: Several techniques: (1) class_weight parameter in loss function (weight minority class higher); (2) Oversampling minority class (SMOTE for text is tricky); (3) Undersampling majority class (lose data); (4) Use F1-score instead of accuracy for evaluation (accuracy is misleading when imbalanced); (5) Focal loss (penalizes easy examples less, focuses on hard ones). My IMDB dataset is perfectly balanced (25K/25K) so this wasn't needed, but I know how to handle it.

Q45: If someone gives you a review in bullet points vs paragraphs, would the model handle both?

A: Yes, because after preprocessing, both become flat sequences of tokens. Bullet points, paragraphs, single sentences - all get cleaned, tokenized, and padded to 300. The model sees only the word vectors, not the formatting. However,

very short bullet-point reviews might have more padding (zeros) which slightly dilutes the signal. The CNN's max pooling handles this well - it picks the strongest signal regardless of how much padding surrounds it.

=====

TIPS FOR THE INTERVIEW

=====

1. START WITH THE COMPARISON ANGLE: "I compared 5 deep learning architectures and found CNN is best for sentiment analysis because..."
2. KEY NUMBERS TO MEMORIZE:
 - Dataset: 50,000 reviews, balanced, 80/20 split
 - Word2Vec: 300 dimensions, Google News, 3M vocabulary
 - Best model: CNN at 89.02%
 - All models: 87-89% (shows embeddings matter more than architecture)
 - Sequence length: 300 tokens
 - Preprocessing: 7 steps (contractions, HTML, lowercase, special chars, stopwords, lemmatization, padding)
3. EXPLAIN WHY CNN BEATS LSTM: "Sentiment is determined by local phrases like 'waste of time' or 'loved it'. CNN's convolutional filters are designed to detect exactly these local patterns."
4. TRANSFER LEARNING ANGLE: "Word2Vec trained on 100 billion words - that's transfer learning. My 40K training reviews benefit from knowledge learned on a much larger corpus."
5. KNOW LIMITATIONS: "Static embeddings can't handle polysemy or sarcasm. BERT would solve this but is beyond project scope."

=====

END OF DOCUMENT

=====